CoAP Extensions for Asynchronous Task Resources

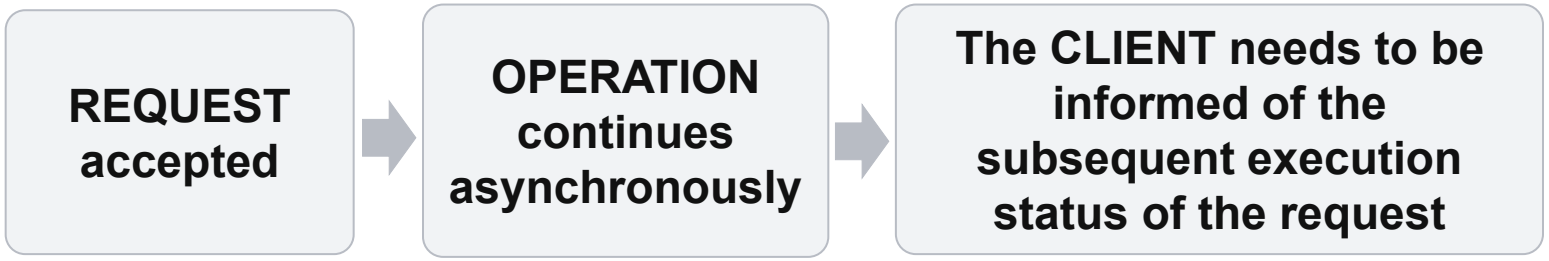
draft-li-coap-task-resources-01

Linzhe Li · Beijing Zhongguancun Laboratory
Yong Cui · Tsinghua University

Why CoAP needs an asynchronous task contract

A request usually returns the immediate result after the application method has been executed, but some operations will continue to run after the request is accepted.

- Firmware installation
- Diagnostics
- Commissioning
- Multi-step complex tasks



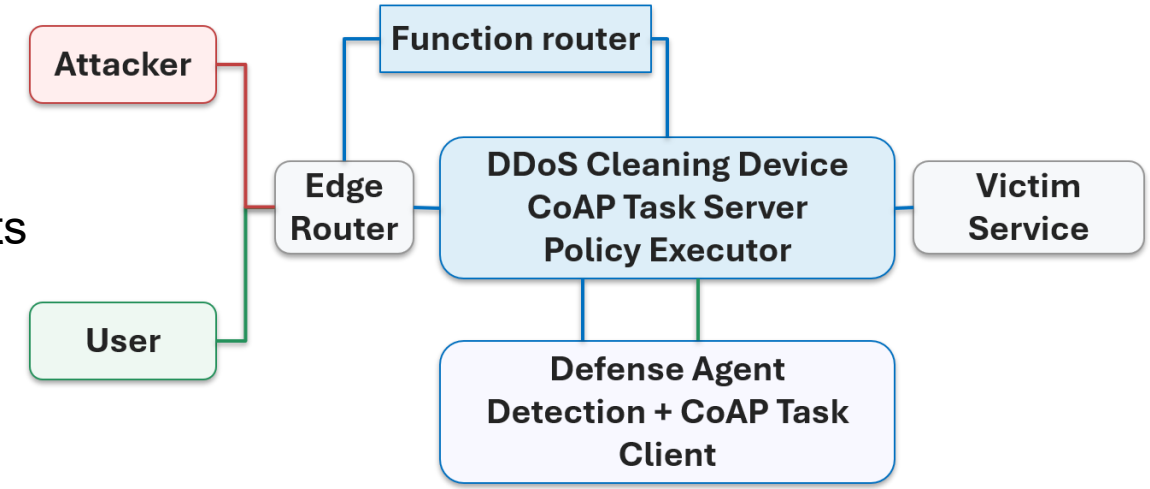
Applications are usually defined by themselves:

Identity: proprietary job IDs and endpoints
State: polling/Observe resources and private vocabularies
Control: cancellation, retry correlation, and retention rules

The pattern is common, but the contract is application-specific.

CoAP-Based DDoS Mitigation Task

```
Creat task payload: {
1: [{                              ; sub-task list
  1: "announce",                   ; BGP operation
  2: 2,                            ; AFI = IPv6
  3: 133,                          ; SAFI = FlowSpec
  4: {                             ; FlowSpec NLRI components
       1: "2001:db8:100::/48",     ; destination prefix
       3: 17,                      ; next header = UDP
       5: 53                       ; destination port = 53
     },
  5: {1: "discard"                 ; traffic action},
  6: 300                           ; rule lifetime: 300 seconds
},
{…}],
```



Deployment has been made in the DDoS security field(NSfocus). Customize the task list according to requirements for task assignment, and stipulate that tasks should be executed in sequence. The task execution status is observed using private domain definitions.

The Scope of This Draft

This document defines a reusable CoAP framework for representing asynchronous and long-running operations as addressable Task Resources.

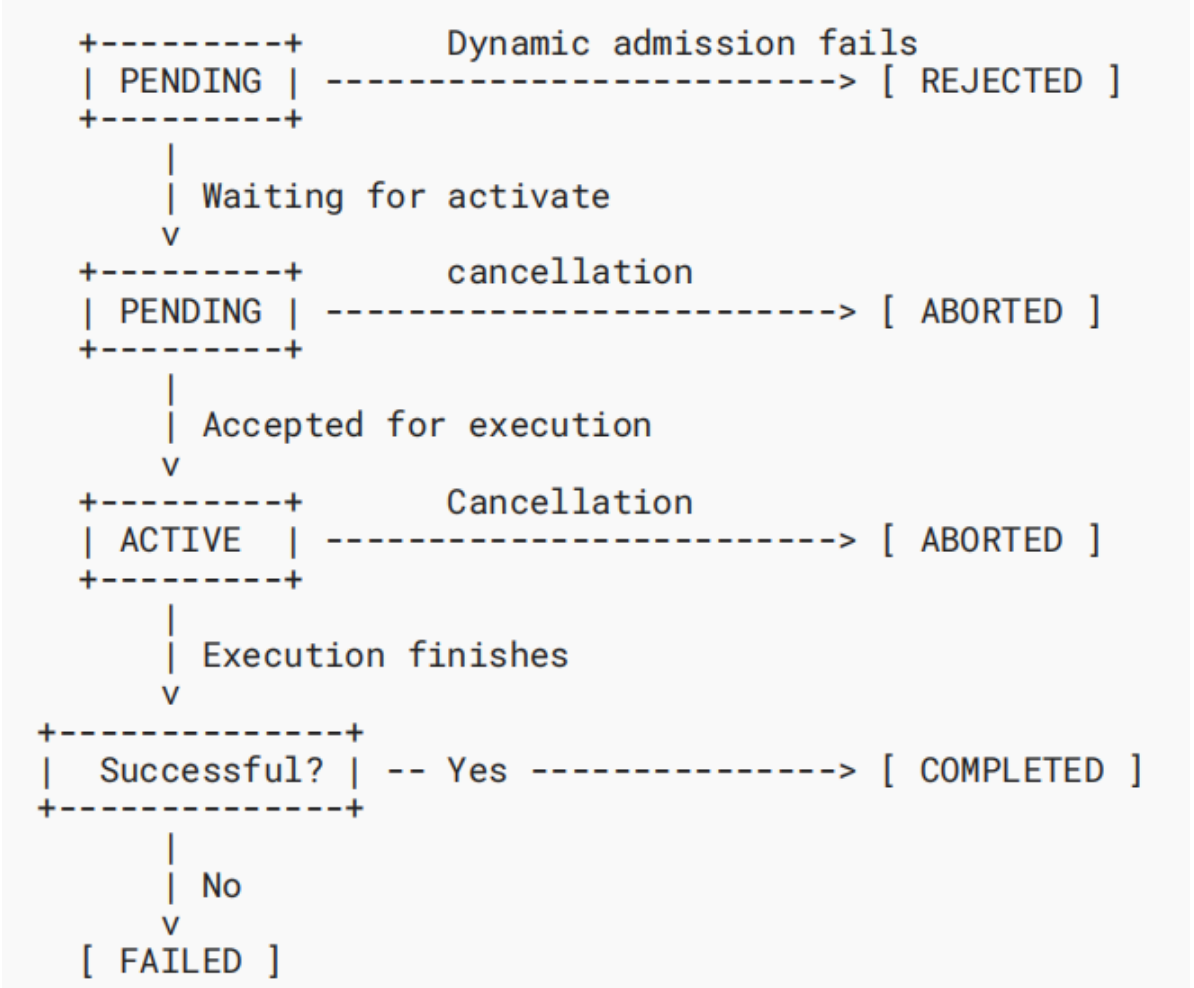
- Lifecycle: common states and terminal retention
- Interaction: creation, retrieval, observation, update, cancellation, deletion
- Discovery: CoRE Link Format with if="core.task"
- Request model: Resource content plus Task controls

The draft reuses core CoAP methods and response codes from RFC 7252 and RFC 8132, Observe and CoRE Link Format from RFC 7641 and RFC 6690, application/multipart-core, CBOR, and CDDL from RFC 8710, RFC 8949, and RFC 8610, and conditional attributes to reduce unnecessary notifications.

Task Lifecycle

- PENDING: The Task Resource has been created, but the execution has not yet started;
- ACTIVE: The operation is in progress;
- COMPLETED: Successfully completed;
- FAILED: Failed to complete after the execution started;
- ABORTED: Cancelled before completion;
- REJECTED: The Task Resource has been created, but the dynamic approval before execution failed.

The final state task should be kept for a sufficient period of time for the client to read; after the retention period ends, subsequent access will return 4.04.



Task Request

Use Content-Format 62 (application/multipart-core)

Part	Name	Content
1	Resource Content-Format	The registered CoAP Content-Format identifier for Part 2
2	Resource representation	A byte string containing resources and values to be operated on
3	Task Content-Format	The registered CoAP Content-Format identifier for Part 4
4	Task representation	A byte string containing task controls, or `null` when no Task content is supplied

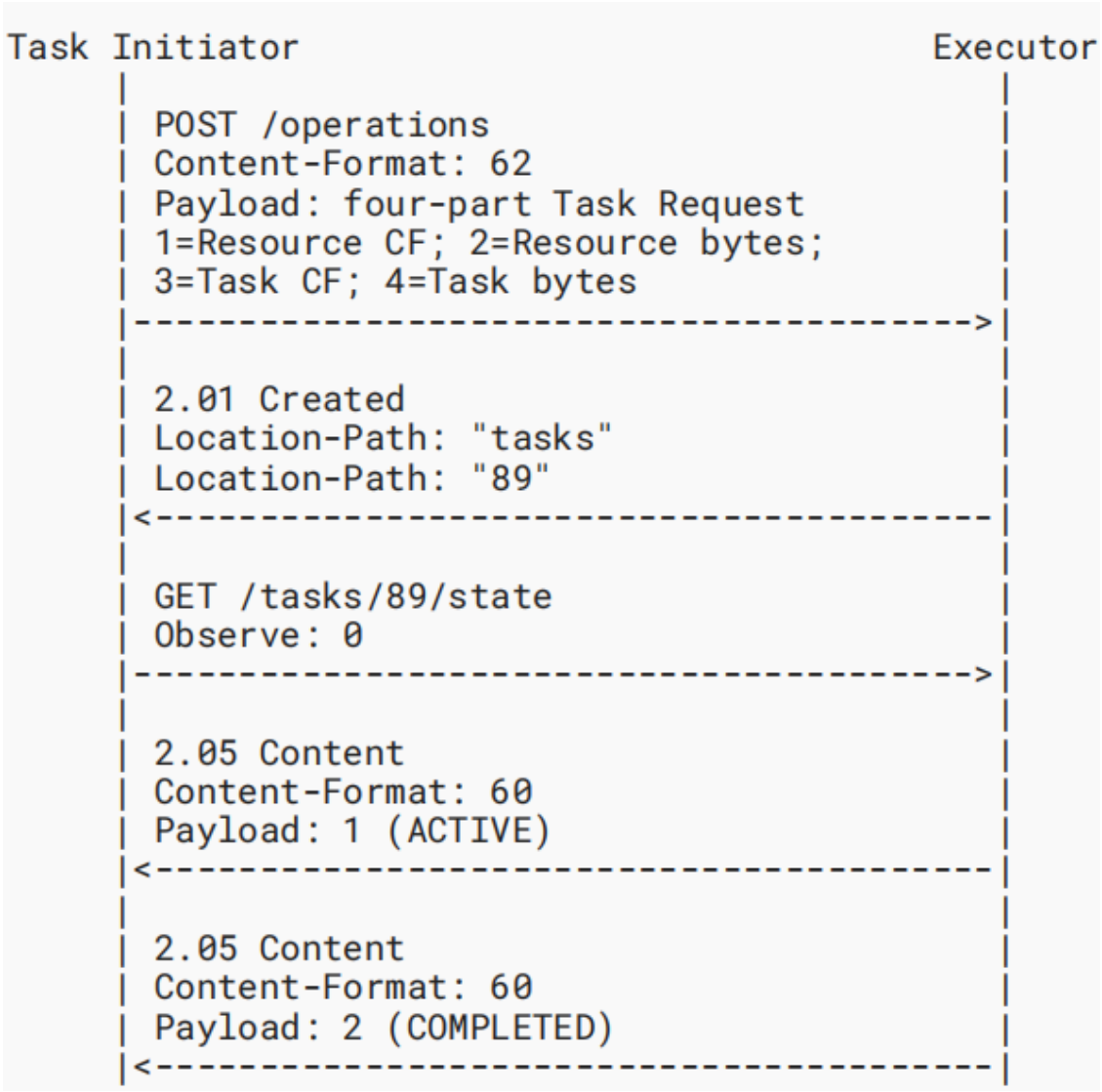
WHAT — target resources, operation order, execute profile, value types and operations are permitted..?

HOW — execution mode, profile-specific task parameters..?

execution mode:

- ATOMIC(1): All-or-nothing guarantee (all resource operations complete atomically)
- SEQUENTIAL (2):Ordered execution + Sub-Results (execute in the order specified by Resource representation; whether to continue upon failure is defined by the profile)

Task Creation and Control



Before create:

- Validate multipart structure and embedded representations
- Check operation and execution-control support
- Authorize every target and enforce profile safety rules

Created:

- new Task Resource URI is returned using Location-Path

Observe:

- A client registers an Observe relationship on the Task Resource or its /state, /progress, or /eta and receives notifications when the value changes.
- The notification payload uses Content-Format 60 (application/cbor)

Value	State
0	PENDING
1	ACTIVE
2	COMPLETED
3	FAILED
4	ABORTED
5	REJECTED

Task Monitoring

- Task status `GET /tasks/89`

```
Task-Status = {
  1 => task-state,           ; task state
  ? 2 => progress,           ; progress percentage
  ? 3 => uint,               ; estimated seconds to completion
  ? 4 => tstr,               ; diagnostic text
  ? 5 => [ + Sub-Result ]    ; per-resource-operation result
}

task-state = 0..5
progress = 0..100
coap-response-code = 0..255

Sub-Result = {
  1 => tstr,                 ; target resource path
  2 => coap-response-code    ; CoAP response code
}
```

- Lightweight observe

/tasks/89/state
/tasks/89/progress
/tasks/89/eta

Open Issues

- Is this a useful common abstraction for CoAP?
- Are there existing deployments or models we should align with?
- Are there any other existing mechanisms that can be reused?
- Are the lifecycle states and boundaries correct?

Welcome feedback ☺

Contact: lilz@zgclab.edu.cn